Week-3:

Aim: Develop a simple data link layer that performs flow control using the sliding window protocol, and loss recovery using the Go-Back-N mechanism.

Code:

```
#include <stdio.h>
int main()
{
   int totalFrames, windowSize;
   int i, j;
   int lostFrame;

   printf("Enter Total Number of Frames: ");
   scanf("%d", &totalFrames);

   printf("Enter Window Size: ");
   scanf("%d", &windowSize);

   printf("Enter Lost Frame Number (-1 if no loss): ");
   scanf("%d", &lostFrame);

   printf("\n----- Sender Side -----\n");

   // Move the sliding window after each transmission
   for(i = 0; i < totalFrames; i += windowSize)
   {
      printf("\nSending Window:\n");

      // Sender transmits all frames in the current window
      for(j = i; j < i + windowSize && j < totalFrames; j++)
      {
         printf("Frame %d Sent\n", j);
```

```
}

printf("\n----- Receiver Side -----\n");

// Check whether the lost frame belongs to the current window
if(lostFrame >= i && lostFrame < i + windowSize)
{
   // Receiver accepts frames before the lost frame
   for(j = i; j < lostFrame; j++)
   {
      printf("Frame %d Received\n", j);
      printf("ACK %d Sent\n", j);
   }

   // Simulate frame loss
   printf("Frame %d Lost\n", lostFrame);

   // Receiver discards out-of-order frames (Go-Back-N)
   for(j = lostFrame + 1; j < i + windowSize && j < totalFrames; j++)
   {
      printf("Frame %d Discarded (Out of Order)\n", j);
   }

   // Timeout occurs because ACK is not received
   printf("\nTimeout Occurred\n");
   printf("Go-Back-N Retransmission Begins\n\n");

   // Retransmit from the lost frame onwards
   for(j = lostFrame; j < i + windowSize && j < totalFrames; j++)
   {
```

```
            printf("Retransmitting Frame %d\n", j);
            printf("Frame %d Received\n", j);
            printf("ACK %d Sent\n", j);
        }
    }
    else
    {
        // Normal transmission when no frame is lost
        for(j = i; j < i + windowSize && j < totalFrames; j++)
        {
            printf("Frame %d Received\n", j);
            printf("ACK %d Sent\n", j);
        }
    }
}

printf("\nAll Frames Successfully Transmitted.\n");

return 0;
}
```

Output: Without any frame loss:

```
Enter Total Number of Frames: 8
Enter Window Size: 4
Enter Lost Frame Number (-1 if no loss): -1

----- Sender Side -----

Sending Window:
Frame 0 Sent
Frame 1 Sent
Frame 2 Sent
Frame 3 Sent

----- Receiver Side -----
Frame 0 Received
ACK 0 Sent
Frame 1 Received
ACK 1 Sent
Frame 2 Received
ACK 2 Sent
Frame 3 Received
ACK 3 Sent

Sending Window:
Frame 4 Sent
Frame 5 Sent
Frame 6 Sent
Frame 7 Sent

----- Receiver Side -----
Frame 4 Received
ACK 4 Sent
Frame 5 Received
ACK 5 Sent
Frame 6 Received
ACK 6 Sent
Frame 7 Received
ACK 7 Sent

All Frames Successfully Transmitted.

--------------------------------
```

With frame loss:

```
Enter Total Number of Frames: 8
Enter Window Size: 4
Enter Lost Frame Number (-1 if no loss): 2

----- Sender Side -----

Sending Window:
Frame 0 Sent
Frame 1 Sent
Frame 2 Sent
Frame 3 Sent

----- Receiver Side -----
Frame 0 Received
ACK 0 Sent
Frame 1 Received
ACK 1 Sent
Frame 2 Lost
Frame 3 Discarded (Out of Order)

Timeout Occurred
Go-Back-N Retransmission Begins

Retransmitting Frame 2
Frame 2 Received
ACK 2 Sent
Retransmitting Frame 3
Frame 3 Received
ACK 3 Sent

Sending Window:
Frame 4 Sent
Frame 5 Sent
Frame 6 Sent
Frame 7 Sent

----- Receiver Side -----
Frame 4 Received
ACK 4 Sent
Frame 5 Received
ACK 5 Sent
Frame 6 Received
ACK 6 Sent
Frame 7 Received
ACK 7 Sent

All Frames Successfully Transmitted.

--------------------------------
```

Theory:

The **Sliding Window Protocol** is a **flow control protocol** used in the **Data Link Layer** of the OSI model. Its main purpose is to control the rate of data transmission between the sender and the receiver. Instead of sending one frame and waiting for its acknowledgment, the sender is allowed to transmit **multiple frames continuously** before receiving acknowledgments. The number of frames that can be transmitted at a time is called the **window size**. After transmitting all the frames in the current window, the sender waits for acknowledgments from the receiver. Once the acknowledgments are received, the sender moves the window forward and transmits the next set of frames. This process is known as **sliding the window**, which improves network efficiency by utilizing the communication channel effectively.

To provide reliable communication, the **Go-Back-N Automatic Repeat Request (ARQ)** mechanism is used for **error and loss recovery**. In this mechanism, the receiver accepts only the frames that arrive in the correct order. If a frame is lost or damaged during transmission, the receiver acknowledges only the correctly received frames before the lost frame and discards all subsequent out-of-order frames. Since the acknowledgment for the lost frame is not received, the sender waits for a predefined timeout period. After the timeout expires, the sender **retransmits the lost frame and all the remaining frames in the current window**. This process is called **Go-Back-N**, as the sender goes back to the lost frame and retransmits all subsequent frames to ensure reliable data delivery.

In this program, the total number of frames, window size, and the frame to be lost are provided by the user. The sender transmits frames according to the specified window size. If no frame is lost, the receiver successfully receives all frames and sends acknowledgments. If a frame is lost, the program simulates the Go-Back-N mechanism by discarding all subsequent frames in that window, generating a timeout, and retransmitting the frames from the lost frame onward. Thus, the program demonstrates both **flow control using the Sliding Window Protocol** and **loss recovery using the Go-Back-N ARQ mechanism** in a simple and easy-to-understand manner.